

System Design da Memória em Sistemas Operacionais

Texto de Apoio

Gean Feitosa Cavalcante

2026

1. Introdução e enquadramento do problema

A memória é o recurso que sustenta toda execução em um sistema computacional. Sem ela não há instruções para a CPU buscar, nem espaço para os dados manipulados pelos processos. Apesar dessa centralidade, é também o subsistema mais difícil de raciocinar: opera em escalas que vão de nanossegundos (cache) a milissegundos (swap), serve simultaneamente o hardware, o kernel e o usuário, e exige isolamento absoluto entre processos sem perder eficiência.

Este trabalho responde à pergunta central do enunciado — *“como a memória é usada do início ao fim por hardware, processador, kernel, sistema operacional e processos, garantindo desempenho, isolamento, proteção e continuidade da execução?”* — por meio de um System Design composto por onze diagramas visuais e três análises críticas. A escolha de tratar memória como um *sistema* (em vez de “RAM”) é deliberada: só uma visão integrada deixa claro que cada camada existe para resolver um problema das outras.

2. Decisões do projeto

2.1 Estrutura em camadas

O diagrama macro segue a cadeia exata pedida no escopo: **Hardware → CPU → MMU/TLB → Kernel → Gerência de Memória do SO → Processos → Ferramentas de Observação**. Essa ordem foi mantida como esqueleto narrativo do trabalho inteiro, porque ela já reflete o “caminho de um acesso” — entendendo a cadeia, o leitor compreende todas as outras seções.

2.2 Convenção de cinco campos por bloco

Cada bloco visual carrega cinco informações: **nome, função, entrada, saída, risco/limitação**. Essa convenção, exigida pelo escopo, força a tratar cada componente como uma “caixa preta com interface” — abordagem típica de documentação de arquitetura de sistemas. Em particular, o campo

risco evita que o trabalho se resuma a descrever o “caminho feliz” e cobra mostrar como cada peça pode falhar.

2.3 Paleta de cores semântica

Foram adotadas as seis cores sugeridas pelo escopo (azul para hardware/CPU, roxo para tradução, verde para userspace, vermelho para proteção/falha, laranja para kernel, cinza para secundário/swap). A paleta funciona como uma **legenda implícita**: ao olhar qualquer diagrama, o leitor identifica imediatamente que tipo de componente é cada bloco.

2.4 Conteúdo modular reaproveitável

Cada arquivo `.md` da pasta `conteudo/` cumpre duplo papel: alimenta os blocos do design visual e fornece a base teórica resumida no texto de apoio. Esse encadeamento foi escolhido para evitar redundância: a teoria sustenta o visual, e o visual sintetiza a teoria.

3. Conceitos centrais utilizados

Quatro conceitos atravessam o trabalho inteiro:

Memória virtual — o processo nunca toca o endereço físico; cada acesso passa pela MMU. Esse fato isolado libera proteção, *sharing* e *swap*, e é a base sobre a qual tudo se monta. Sem memória virtual, isolamento entre processos seria impossível em sistemas multiusuário.

Paginação — espaço virtual e físico divididos em blocos de tamanho fixo (4 KiB no Linux x86-64). Resolve fragmentação externa, permite alocação não-contígua, suporta *swap* por granularidade fina, e torna o *Copy-on-Write* prático. As seções 6, 7 e 8 do conteúdo examinam, respectivamente, por que ela venceu a alocação contígua, qual fragmentação ela introduz no lugar da externa, e como funciona em detalhe.

Proteção por hardware — a MMU verifica os bits da PTE em cada acesso. Sem essa verificação ser feita em silício, o custo seria proibitivo. A seção 5 mostra como essa proteção, somada à separação de modos usuário/kernel, é o que torna possível multiusuário moderno.

Page cache — a grande “invenção operacional” do Linux moderno: usar a RAM ociosa como cache transparente do sistema de arquivos. A seção 9 mostra que esta é, na prática, a maior consumidora de RAM em um Linux saudável — e que é elástica, cede sob pressão. Ignorar isso leva ao erro clássico do operador ao olhar `free` e achar que o sistema está “sem memória”.

4. Relação entre visual e teoria

Cada diagrama do design tem correspondência direta com uma seção do conteúdo escrito:

Diagrama visual	Conteúdo escrito
01 — Diagrama Macro	Seção 1: cadeia HW→observação
02 — Hierarquia de Memórias	Seção 2: registradores → cache → RAM → swap
03 — Fluxo de Tradução	Seção 3: passo a passo do acesso
04 — Mapa do Processo	Seção 4: texto, dados, heap, stack, mmap
05 — Proteção e Isolamento	Seção 5: PTE, modos kernel/usuário, COW
06 — Técnicas de Gerenciamento	Seção 6: seis técnicas comparadas
07 — Fragmentação	Seção 7: interna vs externa
08 — Paginação Visual	Seção 8: páginas, frames, page fault
09 — Linux na Prática	Seção 9: zonas, page cache, OOM
10 — Observabilidade	Seção 10: free, top, vmstat, /proc
11 — Conclusão Visual	Análise crítica e síntese

As três análises críticas (*trade-offs*, erros e riscos, conclusão crítica) não têm diagrama dedicado mas amarram visualmente os blocos anteriores: o *trade-off* “contígua vs paginação” referencia os diagramas 6 e 7; “*swap* excessivo” da análise de riscos cita o diagrama 3 de fluxo.

5. Como o Linux comprova na prática

Um dos principais argumentos do trabalho é que **o modelo proposto não é teórico**: ele está implementado, observável e mensurável em qualquer Linux. As correspondências concretas:

- **Tradução de endereço (diagrama 3)** — verificável em `perf stat -e dTLB-load-misses` durante a execução de uma aplicação
- **Mapa do processo (diagrama 4)** — visível em tempo real em `cat /proc/<pid>/maps` para qualquer PID
- **Bits de permissão (diagrama 5)** — refletidos nas colunas `r/w/x/p` do mesmo arquivo
- **Paginação sob demanda (diagrama 8)** — comprovável: ao executar um programa grande, `mmap -X <pid>` mostra que `RSS << VIRT` no início, e cresce conforme uso
- **Page cache (diagrama 9)** — observável: rodar `cat /grande_arquivo > /dev/null` aumenta `Cached` em `/proc/meminfo`
- **Swap (diagrama 9)** — disparável: alocar memória até estourar e ver `vmstat 1` reportar `si/so > 0`
- **OOM killer (diagrama 9)** — visível em `dmesg` após teste de pressão

Cada conceito do design tem um experimento simples que o demonstra. O trabalho não é uma descrição de “como deveria ser” — é uma descrição de “como é”.

6. Trade-offs e análise crítica

Toda decisão de design de memória resolve um problema e **cria outro**. Cinco tensões fundamentais foram analisadas no conteúdo detalhado:

1. **Desempenho vs Proteção** — o Linux sempre fica do lado da proteção, mesmo quando isso custa centenas de ciclos em cada syscall ou troca de contexto.
2. **Simplicidade vs Flexibilidade** — flexibilidade vence, ao custo de milhares de páginas de documentação técnica. A alternativa seria não rodar cargas reais.
3. **Memória Contígua vs Paginação** — paginação venceu há quatro décadas. Apenas DMA legado, *huge pages* e NUMA local ainda exigem contiguidade.
4. **Menos Fragmentação vs Maior Overhead** — Linux usa páginas de 4 KiB por padrão, com *Transparent Huge Pages* automáticas para regiões grandes.
5. **Observabilidade Simples vs Detalhada** — começar com `free` e `top`; ir fundo apenas quando o básico não explica.

Os principais riscos operacionais identificados foram: fragmentação (interna e externa), *swap* excessivo (*thrashing*), overhead de paginação por *TLB miss*, mau uso de cache (`drop_caches` em produção), falhas de proteção, crescimento descontrolado de *heap*, *stack overflow* e leitura incorreta de métricas (confundir `buff/cache` com memória perdida).

7. Resposta à pergunta central

A pergunta do escopo é respondida em três níveis:

No nível mais concreto (diagrama 3): um acesso à memória começa com um endereço virtual emitido pela CPU; passa pela MMU para tradução; consulta a TLB e, se preciso, a tabela de páginas; resulta em endereço físico ou em *page fault* tratado pelo kernel. Esse ciclo se repete bilhões de vezes por segundo.

No nível arquitetural (diagrama 1 e conclusão crítica): a memória é gerenciada por uma cadeia de camadas com responsabilidades distintas — hardware fornece o substrato, MMU traduz e protege, kernel coordena, processos consomem, ferramentas observam. Cada camada existe porque resolve um problema da anterior.

No nível operacional (diagramas 9 e 10): no Linux, isso se concretiza em zonas, *page cache*, *swap*, VMAs, paginação sob demanda, e em ferramentas que expõem cada métrica relevante ao operador.

O sistema é construído para ser *visto* — toda decisão importante é refletida em `/proc` ou em sinais.

8. Conclusão

O trabalho mostra que memória, em sistemas modernos, não é “RAM gerenciada” — é um **sistema** com responsabilidades distribuídas entre hardware, kernel, processos e ferramentas. O modelo que melhor representa SOs modernos combina memória virtual, paginação, proteção em hardware, gerenciamento centralizado pelo kernel, *page cache* elástico e observabilidade rica. Esse modelo venceu porque é o único que entrega simultaneamente isolamento, eficiência, flexibilidade e capacidade de diagnóstico — quatro requisitos inegociáveis em qualquer sistema multiusuário atual.

O design visual deste trabalho não tenta inventar uma representação nova — ele **organiza visualmente o que o Linux já implementa**, conectando teoria, diagrama e ferramenta de modo que cada peça seja localizável tanto em livros de Sistemas Operacionais quanto em uma sessão de terminal.